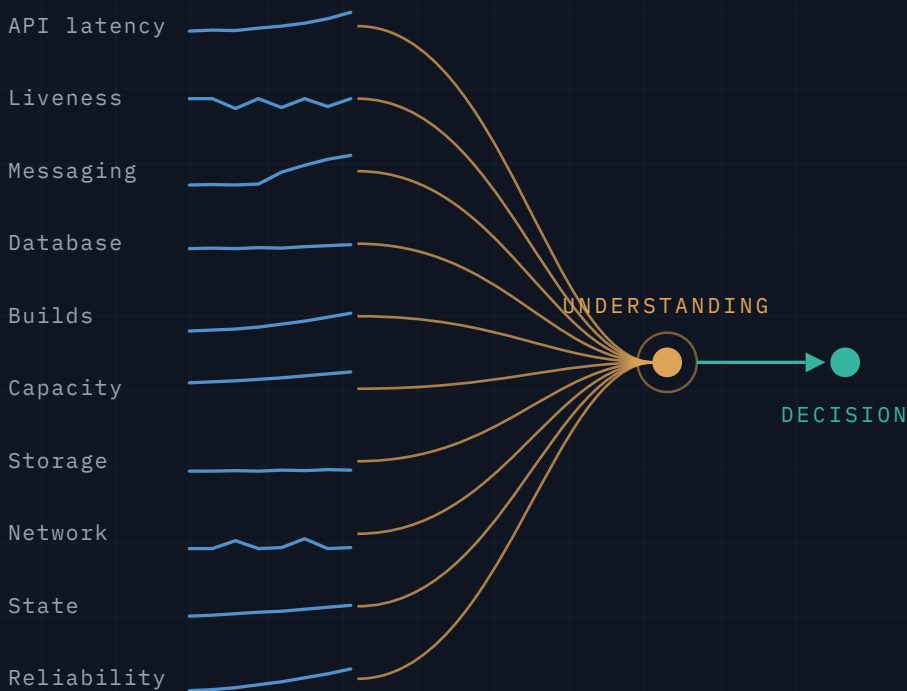


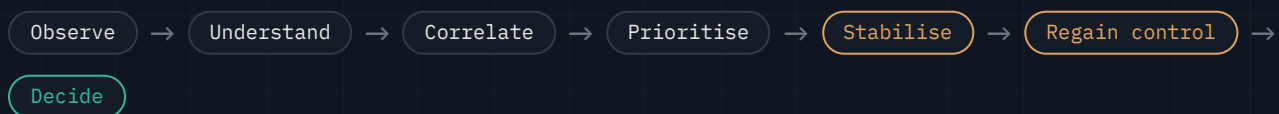
Regaining Control

Observability for OpenStack Platforms in Difficulty

Ten signals, one shared language. A visual guide for technical decision-makers.



THE PATH



Soufian Zaouam

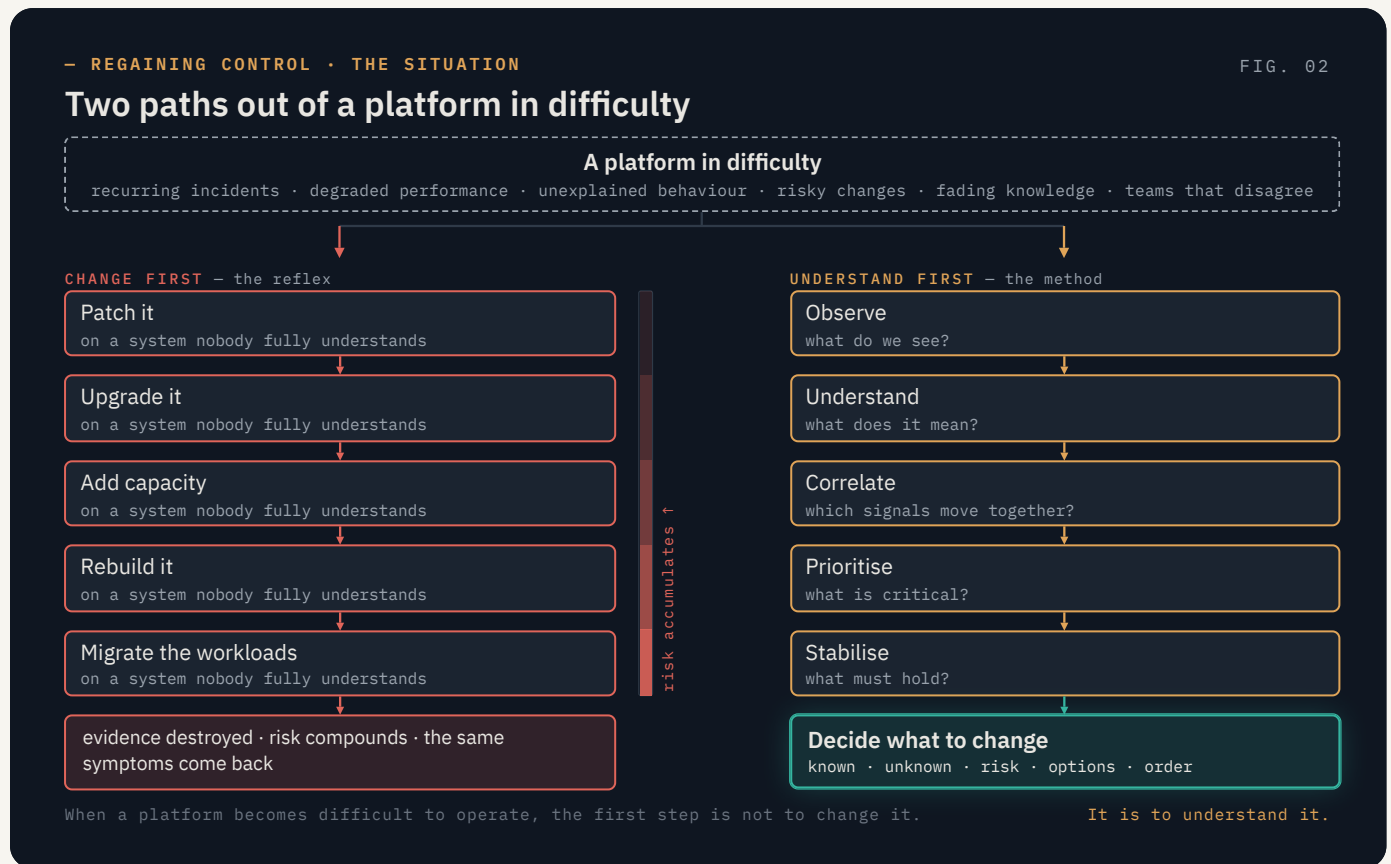
17 pages · 10 minutes to read, 30 to go deeper

Observability is a shared language for understanding operational reality.

Signal → Meaning → Impact → Risk → Decision

When a platform becomes difficult

When a platform becomes difficult to operate, the first step is not to change it. It is to understand it.



It rarely happens at once

Incidents recur. Performance degrades. Some behaviour nobody can explain. A change that used to take an afternoon now takes a week and a rollback plan. The people who built the platform have moved on; the people who run it describe it differently depending on whom you ask.

The reflex is to act on the platform

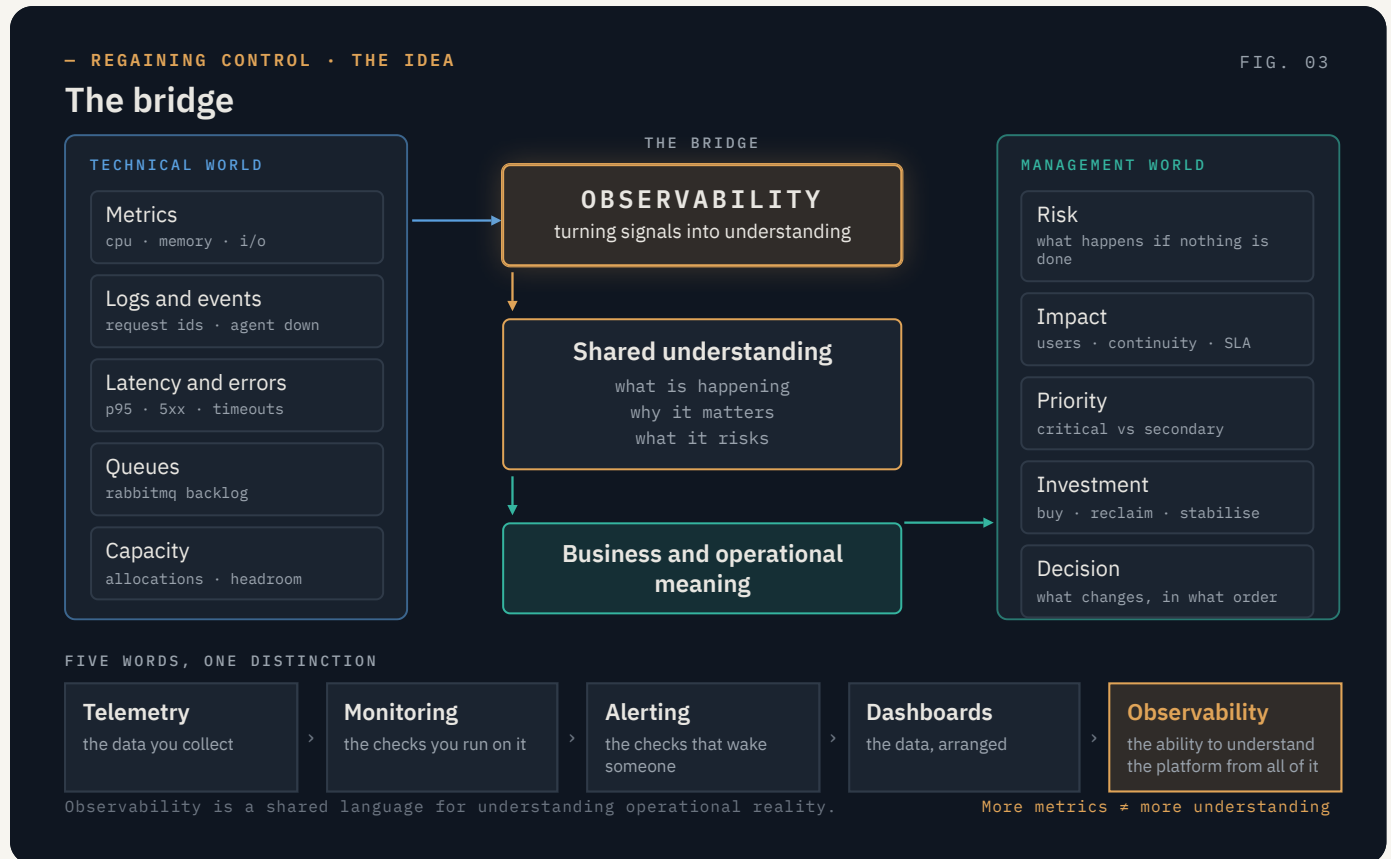
Patch it. Upgrade it. Add capacity. Rebuild it. Migrate the workloads. Each of these is a change to a system nobody fully understands, and each one can destroy the evidence you still need.

The other path costs less than it looks

Before changing a critical platform, regain a reliable understanding of its state: what works, what does not, where the risk is, what depends on what. That is what observability is for, and it is what this guide is about.

Observability is a shared language

Observability is a shared language for understanding operational reality.



Two worlds describe the same platform

Engineers talk about CPU, latency, packet loss, API errors, RabbitMQ queues, database connections, scheduler delays, Ceph health. Decision-makers talk about risk, availability, business impact, continuity, SLAs, cost, priority, investment. Both are right. Neither can act on the other's words.

Observability is the bridge

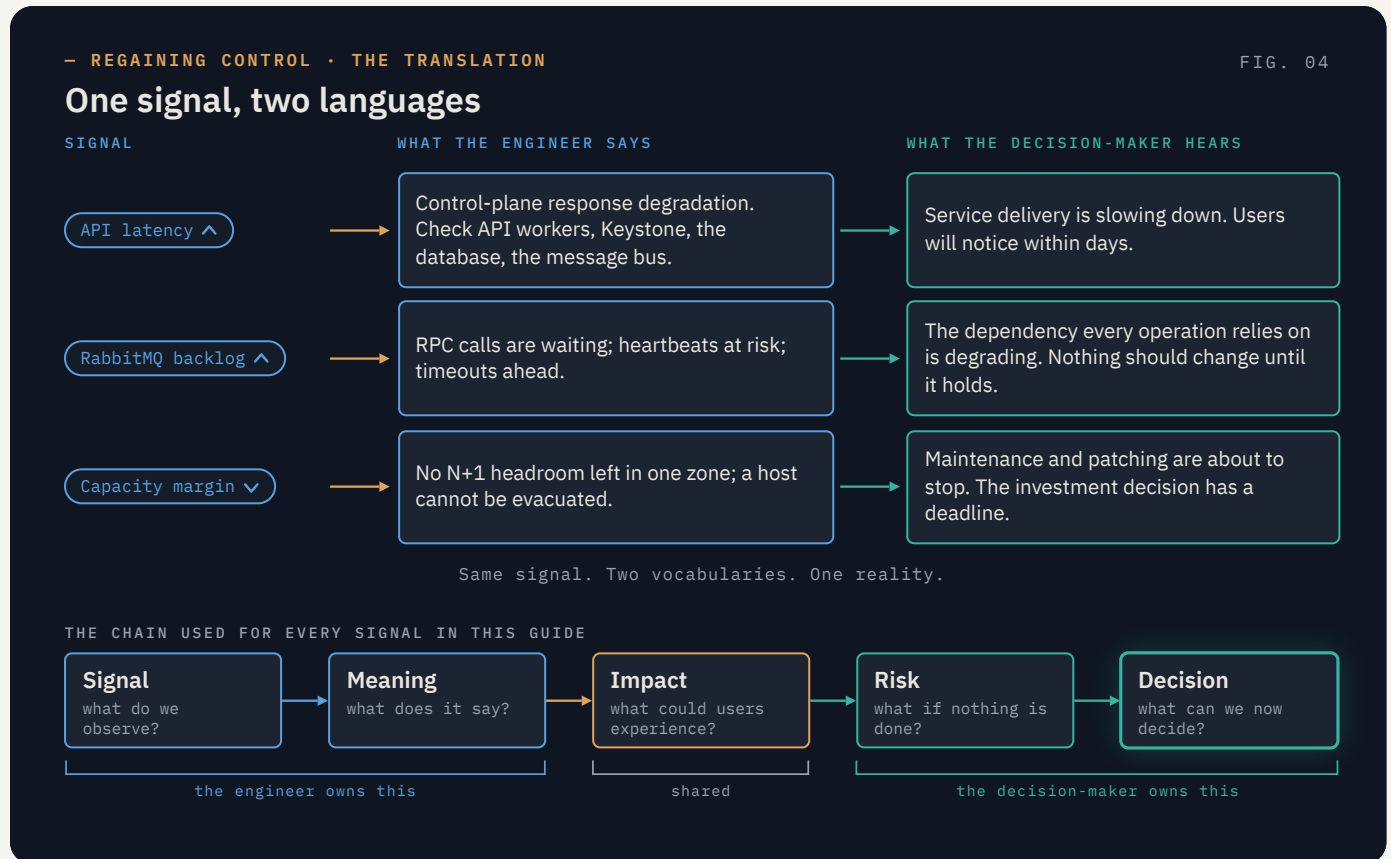
Not the metrics themselves, but the work of turning technical signals into a shared understanding of what is happening, why it matters and what it risks. On that understanding, and only on it, a decision can be made.

Five words, one distinction

Telemetry is the data you collect. Monitoring is the checks you run on it. Alerting is the checks that wake someone. Dashboards are the data, arranged. Observability is the ability to understand the state of the platform from all of it. More metrics do not mean more understanding.

One signal, two languages

Same signal. Two vocabularies. One reality.



Every signal is read through the same chain

Signal: what do we observe?
 Meaning: what does it say, technically and operationally?
 Impact: what could users experience?
 Risk: what happens if nothing is done?
 Decision: what does this information allow us to decide?

The chain is what makes a signal usable by both sides

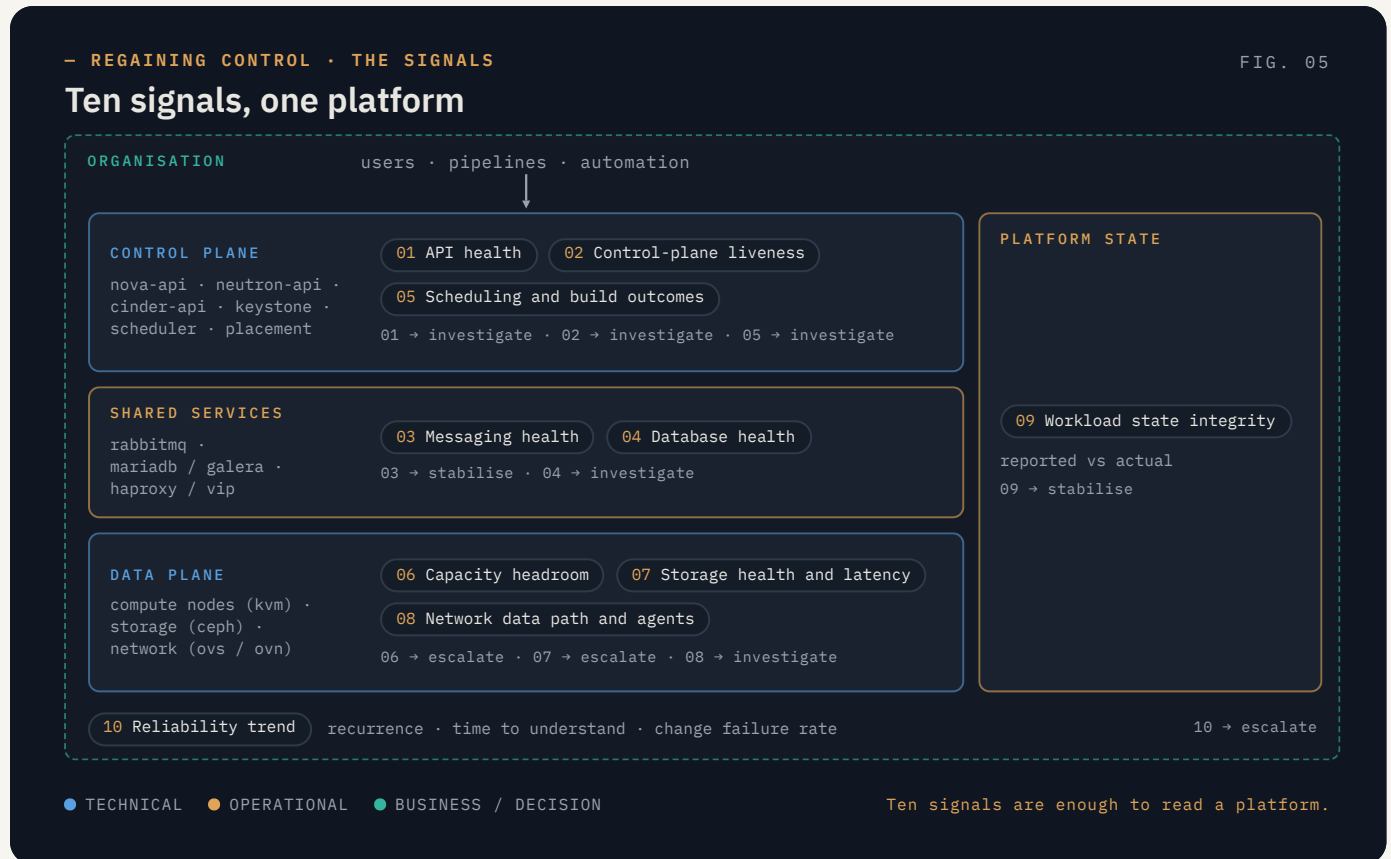
The engineer owns the first two steps, the decision-maker the last two; the middle is shared. When a team cannot walk a signal through the chain, it has data, not understanding.

Ask for the translation, not the metric

A decision-maker does not need the queue depth. They need to know that the dependency every operation relies on is degrading, that users will see it within days, and that nothing should be changed on the platform until it holds. That sentence is the deliverable of observability.

Ten signals, one platform

Ten signals are enough to read a platform. Fifty are enough to stop reading it.



Chosen for the decision they support

Not the most technical, not the most numerous: the signals a decision-maker should be able to ask about, and that an engineer can explain in one sentence. Each one supports a named decision: invest, stabilise, freeze, investigate, escalate.

Nine about the platform, one about the organisation

They sit on three layers: the control plane that receives requests, the shared services every operation depends on, and the data plane where workloads run. One signal cuts across the layers: whether what the platform reports is true. The tenth measures whether the organisation is learning.

Read them together

No signal is read alone; each card names the signals to read with it. Why that matters is on page 11. The signals left out, and the reasons, are in the repository.

API health • Control-plane liveness

Is the platform responding, and is the capacity it reports real?

— SIGNAL 01 / 10

API health

Response time and error rate, per API service

CONTROL PLANE

— MANAGEMENT QUESTION

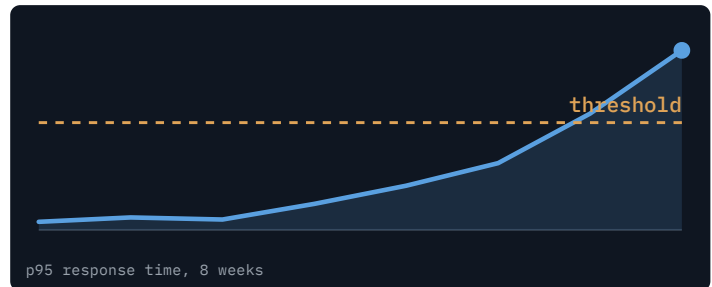
Is this temporary, growing, or becoming a service risk?

WHAT WE SEE

API response times (p95) climbing week after week. Timeouts and 5xx responses appearing on Nova, Neutron, Cinder or Keystone. Automation jobs that used to finish now retry.

WHY IT MATTERS

Slower consoles, failing pipelines, deployments that time out. The platform's capacity to deliver change is shrinking, usually long before anyone calls it an incident.



TECHNICAL MEANING

The request path is degrading somewhere behind the API: saturated API workers, slow token validation, a database under flow control, RPC calls waiting on the message bus. The API is where the problem becomes visible, rarely where it starts.

IF NOTHING IS DONE

Timeouts turn into retries, retries add load, and a slow platform becomes an unavailable one.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Monitor while flat, investigate when trending, freeze changes once users are affected.

read with

Messaging health

Database health

Control-plane liveness

— SIGNAL 02 / 10

Control-plane liveness

Compute services and network agents, as the platform sees them

CONTROL PLANE

— MANAGEMENT QUESTION

Is our capacity unavailable, or invisible?

WHAT WE SEE

Compute services and network agents reported down, or flapping between up and down. Hosts that engineers can log into, and that the platform declares dead.

WHY IT MATTERS

Builds fail with "No valid host" while the racks are half empty; new ports cannot be bound. The capacity you paid for is not the capacity the platform can use.



TECHNICAL MEANING

Liveness is inferred from heartbeats that travel over the message bus and are written to the database. A missing heartbeat means the report is late, not that the host is down. The scheduler trusts the report: a host marked down is excluded from every placement decision.

IF NOTHING IS DONE

Invisible capacity is treated as missing capacity: hardware gets ordered, the real fault stays.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Check the messaging path before any capacity decision.

read with

Messaging health

Scheduling and build outcomes

Capacity headroom

Messaging health • Database health

The two shared dependencies every operation relies on.

— SIGNAL 03 / 10

Messaging health

RabbitMQ queues, consumers and partitions

SHARED SERVICES

— MANAGEMENT QUESTION

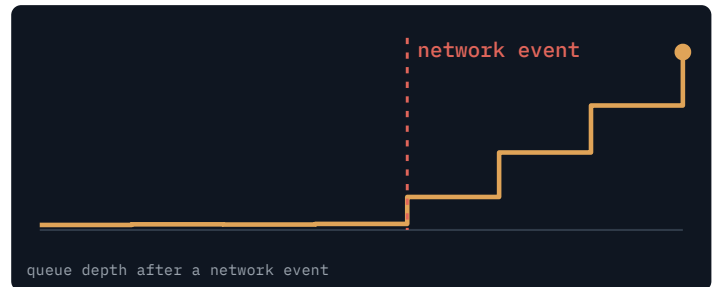
Is a systemic dependency at risk before we change anything else?

WHAT WE SEE

Queues that grow instead of draining. Unacknowledged messages piling up. Consumers disappearing and reconnecting. A cluster reporting a partition after a network event.

WHY IT MATTERS

Instances stuck in BUILD or DELETING, operations that succeed on the second try, unexplained slowness. One shared dependency can take the whole control plane with it. This is the systemic risk of the platform.



TECHNICAL MEANING

OpenStack services talk to each other through RPC over the message bus. When it degrades, calls slow down or time out, heartbeats are lost, and conductor and scheduler build a backlog. Every symptom shows up somewhere else first.

IF NOTHING IS DONE

Degradation spreads to APIs, scheduling and liveness reporting, until the safest option left is to stop the APIs and rebuild the cluster.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Stabilise the bus before any patch, upgrade or capacity change.

read with

API health

Control-plane liveness

Scheduling and build outcomes

— SIGNAL 04 / 10

Database health

MariaDB / Galera cluster state, flow control and growth

SHARED SERVICES

— MANAGEMENT QUESTION

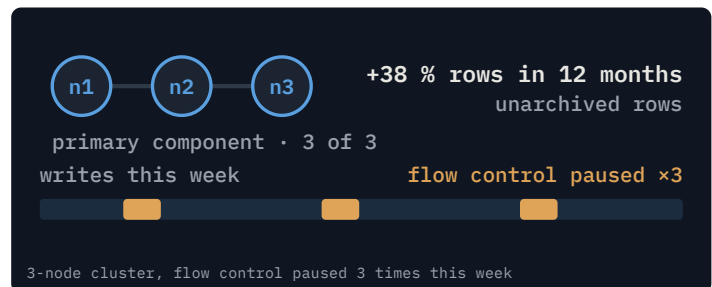
Is the platform's record of what exists trustworthy?

WHAT WE SEE

A cluster that is not Primary, or smaller than it should be. Flow-control pauses. Slow queries and connection pools at their limit. Tables that keep growing because deleted records are never archived.

WHY IT MATTERS

Timeouts, and reported states that do not match reality. If the record is not trustworthy, neither is any report built on it, and every upgrade starts on unstable ground.



TECHNICAL MEANING

Every API write lands here. Under flow control the whole cluster pauses writes so that the slowest node can catch up; exhausted connection pools become API errors; unarchived rows slow queries and lengthen upgrades. This is the platform's record of what exists.

IF NOTHING IS DONE

Split-brain risk, longer recoveries, and upgrades that fail on data nobody cleaned.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Plan hygiene windows before any upgrade programme, not during it.

read with

API health

Messaging health

Workload state integrity

Scheduling and build outcomes • Capacity headroom

Capacity problem, or health problem?

— SIGNAL 05 / 10

Scheduling and build outcomes

Instance builds that succeed, fail or take too long

CONTROL PLANE

— MANAGEMENT QUESTION

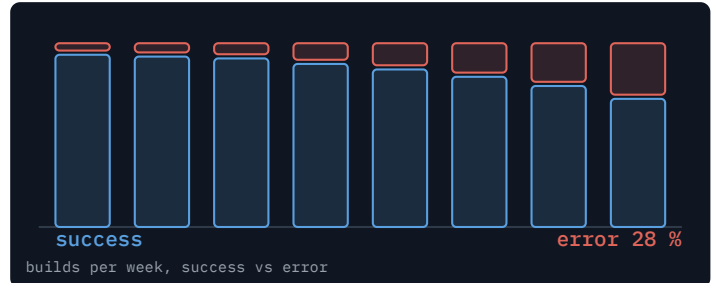
Is this a capacity problem, or a health problem?

WHAT WE SEE

A rising share of builds ending in ERROR. "No valid host was found" in the logs. Time to ACTIVE stretching from seconds to minutes. Port-binding failures and scheduler retries.

WHY IT MATTERS

"I cannot create a VM" is the most visible failure a platform has. Projects stall, and the wrong diagnosis leads to the most expensive wrong decision: buying capacity for a health problem.



TECHNICAL MEANING

Placement and the scheduler could not find a host, or the chosen host could not finish the build. Capacity is the cause only sometimes; just as often it is a host reported down, a network agent that is not binding ports, or a message bus too slow to answer.

IF NOTHING IS DONE

Delivery stops, workarounds multiply, and trust in the platform erodes with every retry.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Correlate with liveness and headroom before deciding to buy, reclaim or repair.

read with

Control-plane liveness

Capacity headroom

Network data path and agents

— SIGNAL 06 / 10

Capacity headroom

Usable margin in zone, and the time left before saturation

DATA PLANE

— MANAGEMENT QUESTION

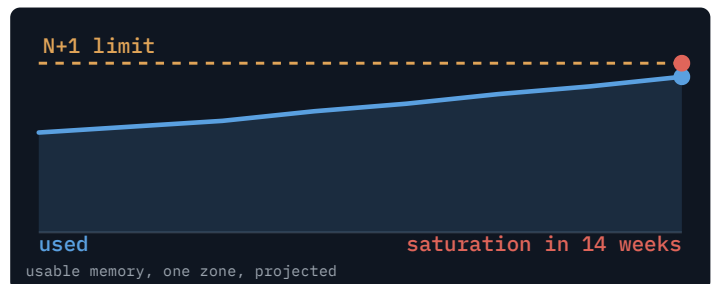
When do we invest, reclaim or freeze onboarding, and can we still patch?

WHAT WE SEE

Allocated versus usable vCPU, memory and disk per availability zone or aggregate, after overcommit and reservations. The margin left to evacuate one host. The trend, and the date it crosses zero.

WHY IT MATTERS

Failed builds, and maintenance windows that keep being postponed. No headroom means no live migration, no patching campaign, no safe upgrade. Capacity is the precondition of maintenance, and therefore of security.



TECHNICAL MEANING

Placement knows the inventory and the allocations; usable headroom is what remains once reserved resources, overcommit ratios and failure domains are taken into account. A cluster can be 60 % allocated and still unable to evacuate a single host.

IF NOTHING IS DONE

Patching stops, security debt grows, and the next hardware failure has nowhere to evacuate to.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

A capacity trend is an investment decision; the date it crosses zero is the deadline.

read with

Scheduling and build outcomes

Workload state integrity

Reliability trend,

Storage health and latency · Network data path and agents

The layers with the widest blast radius.

— SIGNAL 07 / 10

Storage health and latency

Cluster health, fill level and client I/O latency

DATA PLANE

— MANAGEMENT QUESTION

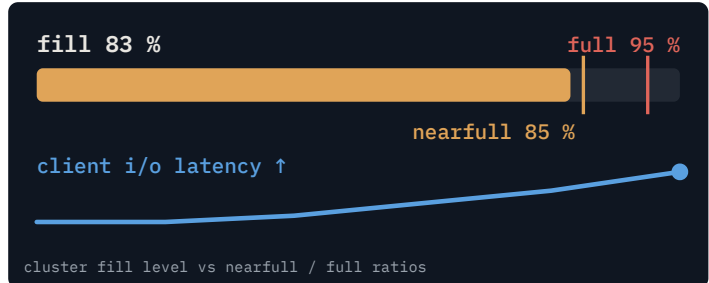
What is our worst-case blast radius, and how far are we from it?

WHAT WE SEE

Health warnings that never clear: OSDs down, placement groups degraded or undersized. Pools approaching their full ratio. Client read and write latency rising. Recovery traffic competing with production I/O.

WHY IT MATTERS

Frozen instances, slow disks, databases inside VMs that time out. The widest blast radius on the platform. A storage incident is a platform incident, and a full cluster is an outage with a long recovery.



TECHNICAL MEANING

Every volume, every image and often every root disk lives on the same shared backend. Degraded placement groups slow or block I/O; a cluster that reaches its full ratio stops accepting writes and instances freeze on I/O errors; recovery can degrade the platform as much as the failure it repairs.

IF NOTHING IS DONE

Fill level and degradation compound until writes stop for everyone at once.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Expansion and rebalancing have lead time; decide before the warning becomes an error.

read with

Workload state integrity

Capacity headroom

API health

— SIGNAL 08 / 10

Network data path and agents

Agents alive, ports bound, packets delivered

DATA PLANE

— MANAGEMENT QUESTION

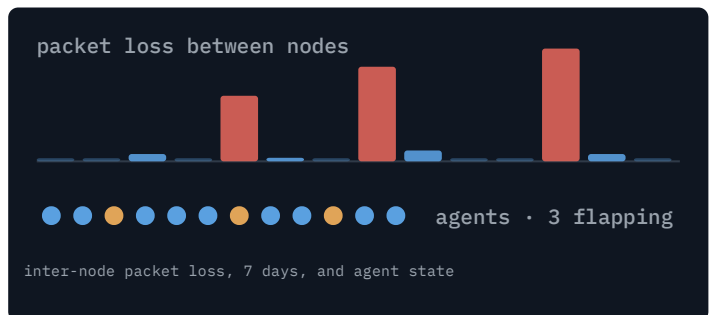
Is it the network layer, or the applications on top of it?

WHAT WE SEE

Network agents (Open vSwitch or OVN, L3, DHCP, metadata) reported dead or flapping. Ports that fail to bind. Packet loss or latency between nodes. Routers failing over more often than expected.

WHY IT MATTERS

"Intermittent connectivity", unreachable new instances, applications blamed for a platform fault. Application availability, and the credibility of the platform team when nobody can explain what happened.



TECHNICAL MEANING

A dead agent does not break existing traffic; it stops new connectivity from being created (ports, addresses, routes). A binding failure turns a build into an ERROR. Router failovers and inter-node loss produce the intermittent problems no single team can reproduce.

IF NOTHING IS DONE

Chronic flakiness: incidents closed without a cause, and users who build around the platform instead of on it.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

Freeze network changes while the layer is unstable; investigate with evidence, not restarts.

read with

Control-plane liveness

Scheduling and build outcomes

Messaging health

Workload state integrity · Reliability trend

Is what the platform reports true? Are we stabilising, or drifting?

— SIGNAL 09 / 10

Workload state integrity

What the platform reports, against what actually runs

PLATFORM STATE

— MANAGEMENT QUESTION

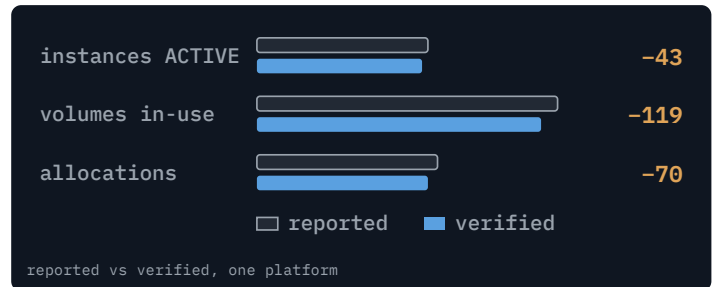
How much of what the platform reports is true?

WHAT WE SEE

Instances in ERROR. Instances stuck in BUILD, DELETING or MIGRATING for hours. Volumes attached to nothing. Ports and allocations that belong to instances which no longer exist. Live migrations that fail or never finish.

WHY IT MATTERS

Resources they cannot use or delete; quotas consumed by ghosts. The share of the platform's reported state you can trust, which is also your upgrade readiness.



TECHNICAL MEANING

The database says one thing, the hypervisor another. Orphaned allocations consume capacity no workload uses; stuck states block users and automation; each inconsistency is a small lie the platform tells about itself, and upgrades fail on those lies.

IF NOTHING IS DONE

Entropy grows until nobody can say what the platform really runs, and every change becomes a gamble.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

A hygiene campaign is cheaper than an upgrade that fails halfway.

read with

Database health

Capacity headroom

Scheduling and build outcomes

— SIGNAL 10 / 10

Reliability trend

Recurrence, time to understand, change failure rate

ORGANISATION

— MANAGEMENT QUESTION

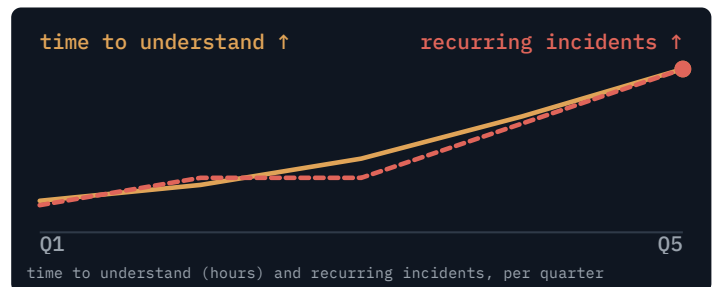
Are we stabilising, or drifting? Do we invest in stability, or in features?

WHAT WE SEE

The same root cause appearing twice. The time between an incident's start and the moment the team can explain it. Incidents closed without a cause. Changes that have to be rolled back. Time to recover.

WHY IT MATTERS

Disruptions that repeat, and confidence that erodes with each one. The trajectory of operational risk, and the credibility of any plan to change the platform.



TECHNICAL MEANING

Not a platform metric: a measure of whether the organisation is learning. A long time to understand means the signals are not readable; recurrence means the cause was never addressed; a high change failure rate means changes are made without understanding.

IF NOTHING IS DONE

Control is lost quietly: each incident is handled, none is understood, and the next change is a gamble.

DECISION CONTEXT

MONITOR

INVESTIGATE

STABILISE

ESCALATE

This is the signal that decides the budget of the other nine.

read with

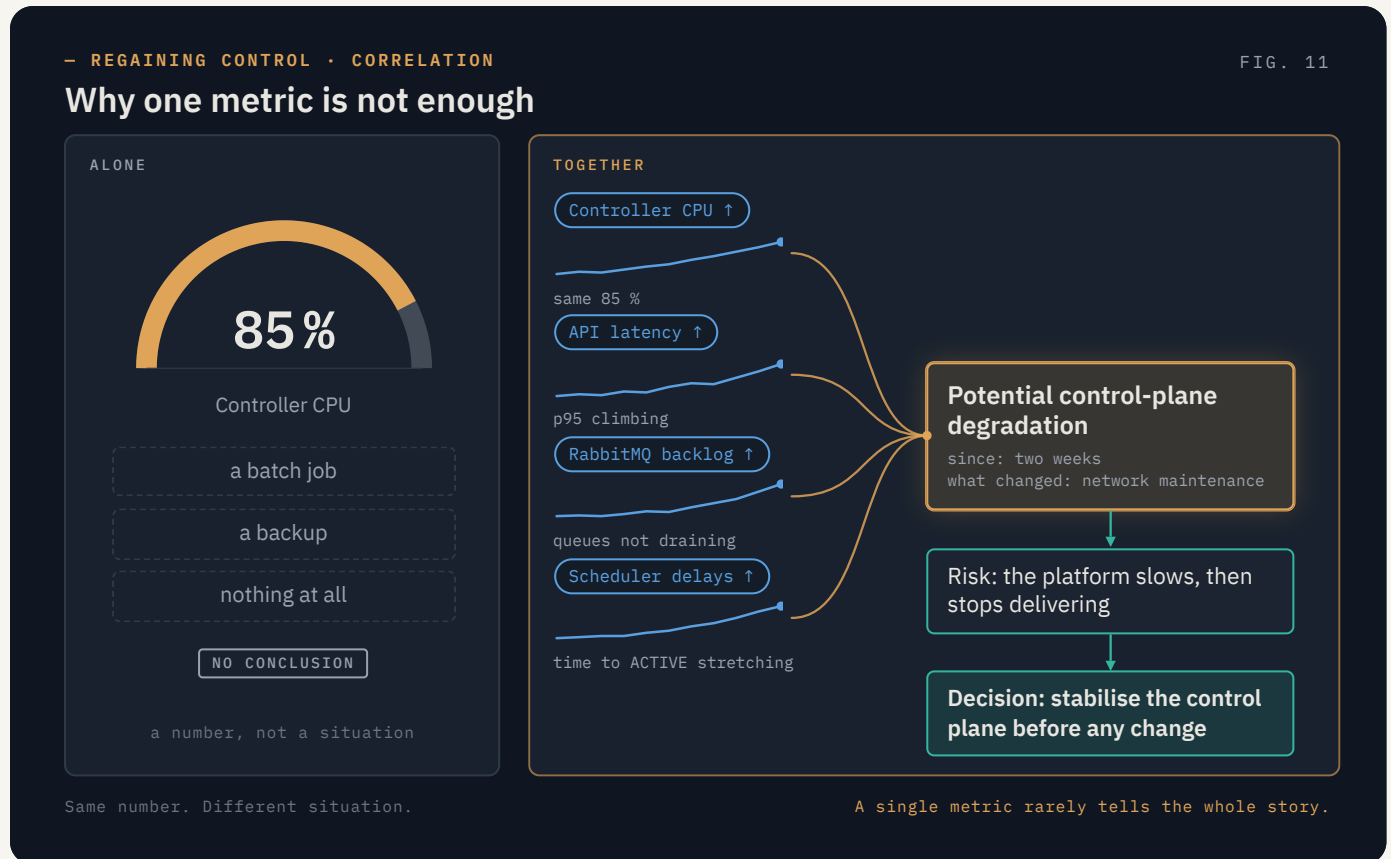
API health

Messaging health

Workload state integrity

Why one metric is not enough

High CPU alone is not a problem. High CPU with three other signals is a story.



A single metric rarely tells the whole story

Controller CPU at 85 % may be a batch job, a backup, or nothing at all. The same CPU, with API latency rising, RabbitMQ queues growing and scheduler delays lengthening, describes a control plane that is saturating. Same number; different situation.

Observability is the work of reconstruction

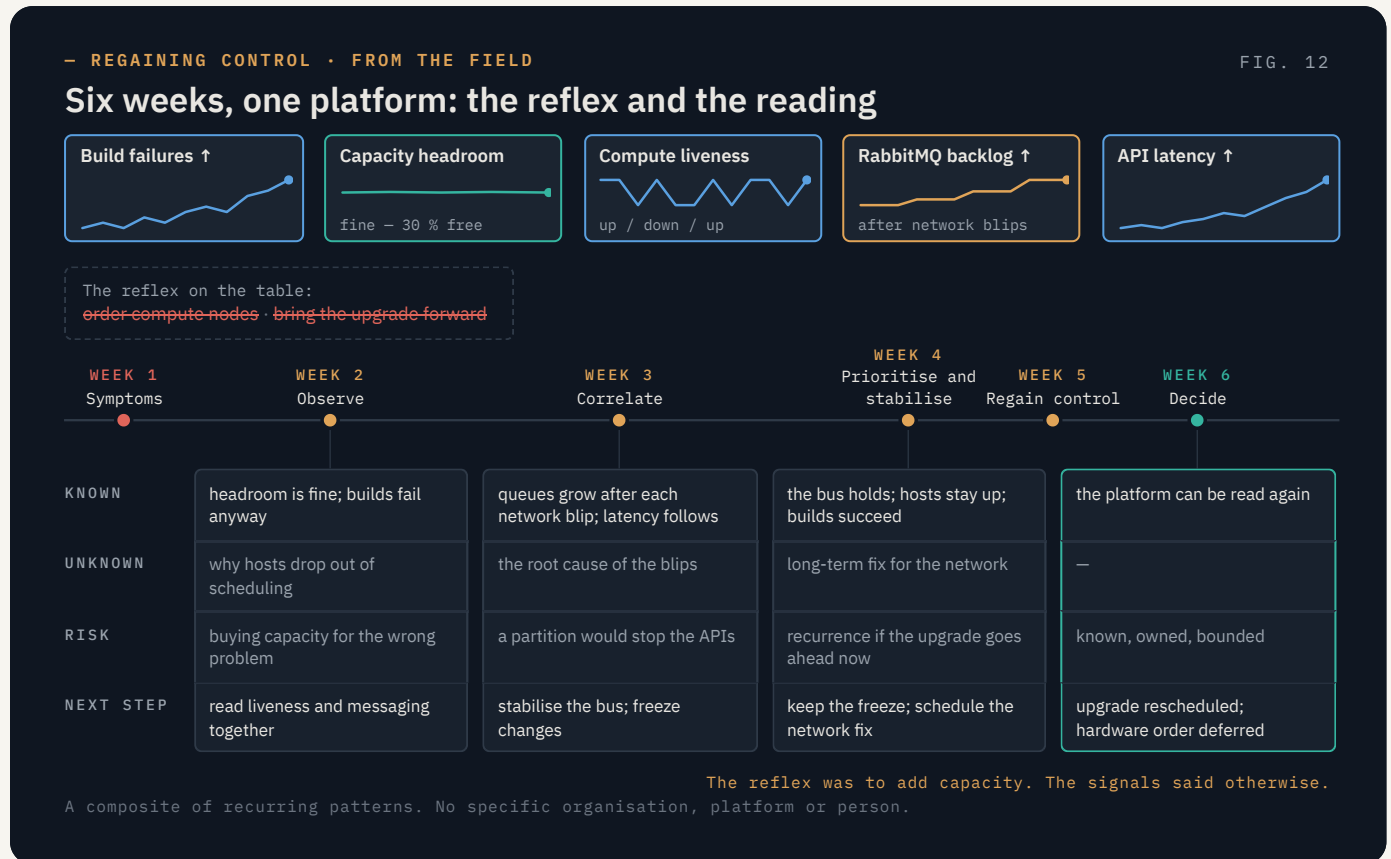
Signals converge into a situation the way symptoms converge into a diagnosis. The question is never "is this metric red?" but "which signals move together, since when, and what changed?"

For a decision-maker, correlation is the difference between a number and a risk

A red metric asks for a reaction. A correlated picture asks for a decision: what is at risk, since when, and what must hold before anything is changed.

A production scenario

The reflex was to add capacity. The signals said otherwise.



The situation

Instance creation on a mid-sized platform slows down over several weeks; some builds fail with "No valid host". Users escalate. Two proposals reach the steering committee within days: order compute nodes, or bring the upgrade forward.

The reading

Capacity headroom is fine; the control plane is not. Several compute services flap between up and down. RabbitMQ queues grow after each network blip; API latency rises with them; the scheduler excludes hosts that are running. The platform is not short of capacity. It is short of a reliable message bus.

The decision

Stabilise messaging first. Freeze the upgrade. Defer the hardware order. What the decision-maker was told at each step is on the timeline: what is known, what is not, what the risk is, what comes next. Six weeks later the upgrade is rescheduled, on a platform that can be read.

A composite of recurring patterns seen in production. No specific organisation, platform or person is described.

From dashboards to decisions

More metrics $\neq$ more understanding. Don't start with dashboards. Start with questions.



A dashboard is not an understanding

Forty panels on a screen tell an engineer where to look and tell a decision-maker nothing. The same platform, seen through ten signals with a state, a trend, a risk and an owner, tells both what to do next.

Two levels, one source

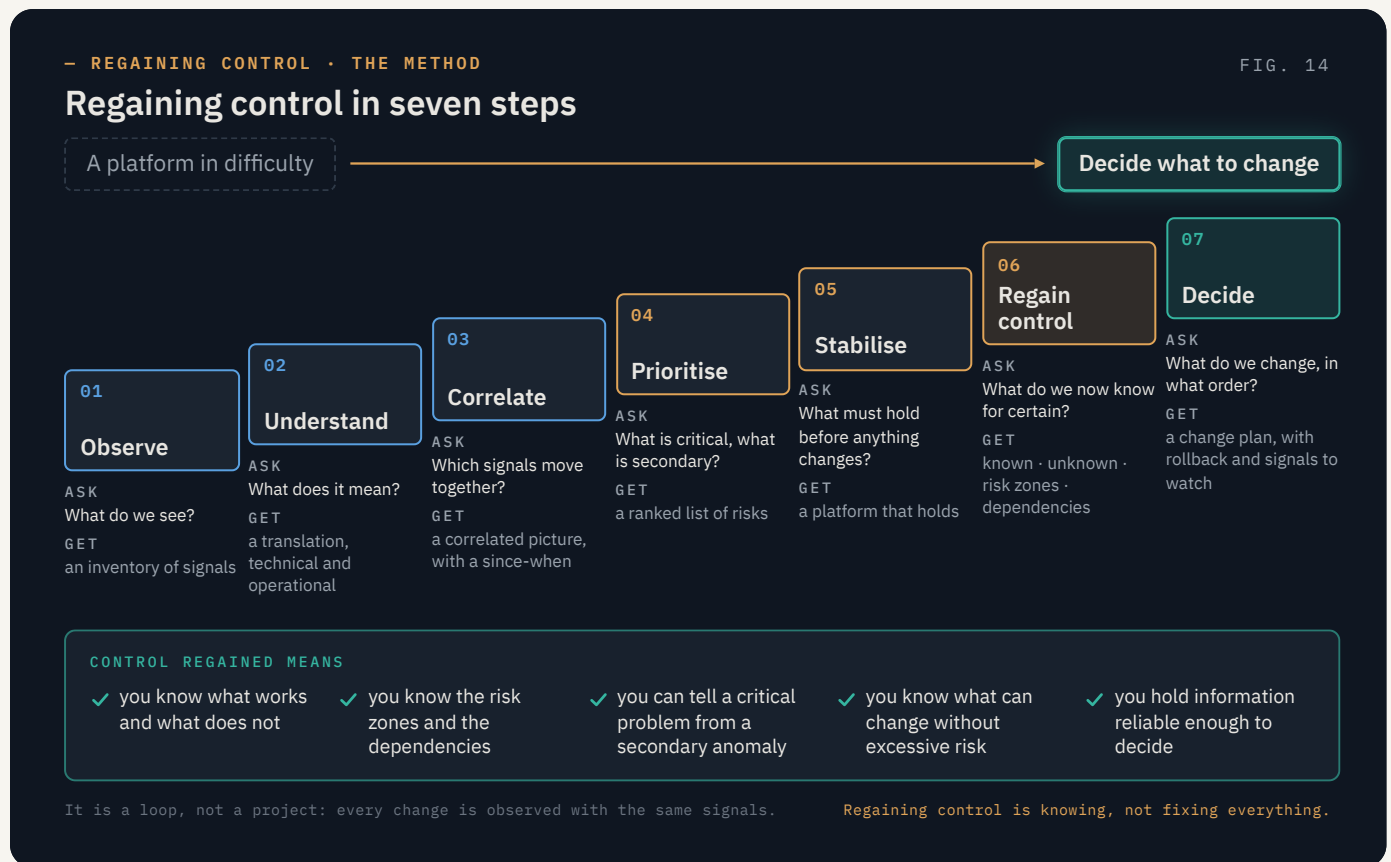
Technical observability is deep, per service, for the people who investigate. Decision-oriented observability is narrow, per question, for the people who decide. They read the same data. The mistake is to show the first to the second.

Five questions before any dashboard

- What do we need to know?
- Why do we need to know it?
- Who needs the information?
- What decision will it support?
- What happens if we don't know?

Regaining control

Regaining control is knowing, not fixing everything.



Seven steps, each with a question the decision-maker can ask

Observe: what do we see?
 Understand: what does it mean?
 Correlate: which signals move together? Prioritise: what is critical, what is secondary?
 Stabilise: what must hold before anything changes? Regain control: what do we now know for certain? Decide: what do we change, and in what order?

Control regained means

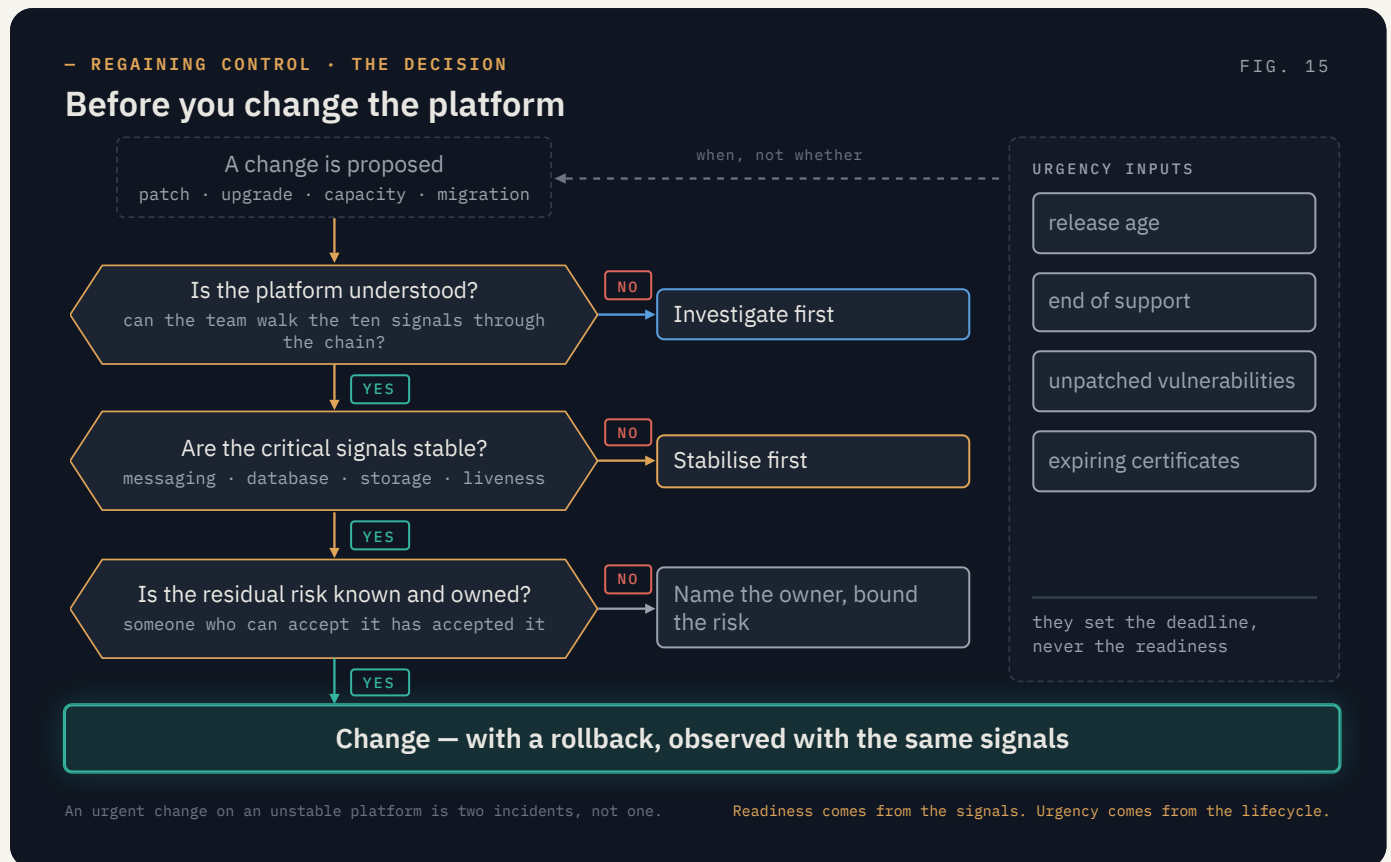
You know what works and what does not. You know the risk zones and the dependencies. You can tell a critical problem from a secondary anomaly. You know what can be changed without excessive risk. And you hold information reliable enough to decide.

It is a loop, not a project

Every change made afterwards is observed with the same signals. A platform under control is not a platform without incidents; it is a platform whose incidents are understood faster than they recur.

Before you change the platform

Readiness comes from the signals. Urgency comes from the lifecycle. A change needs both.



Three questions, in order

Is the platform understood: can the team walk the ten signals through the chain? Are the critical signals stable: messaging, database, storage, liveness? Is the residual risk known, and does someone own it? Three yes: change, with a rollback. A no at the first question: investigate first. A no at the second: stabilise first.

Urgency is a separate input

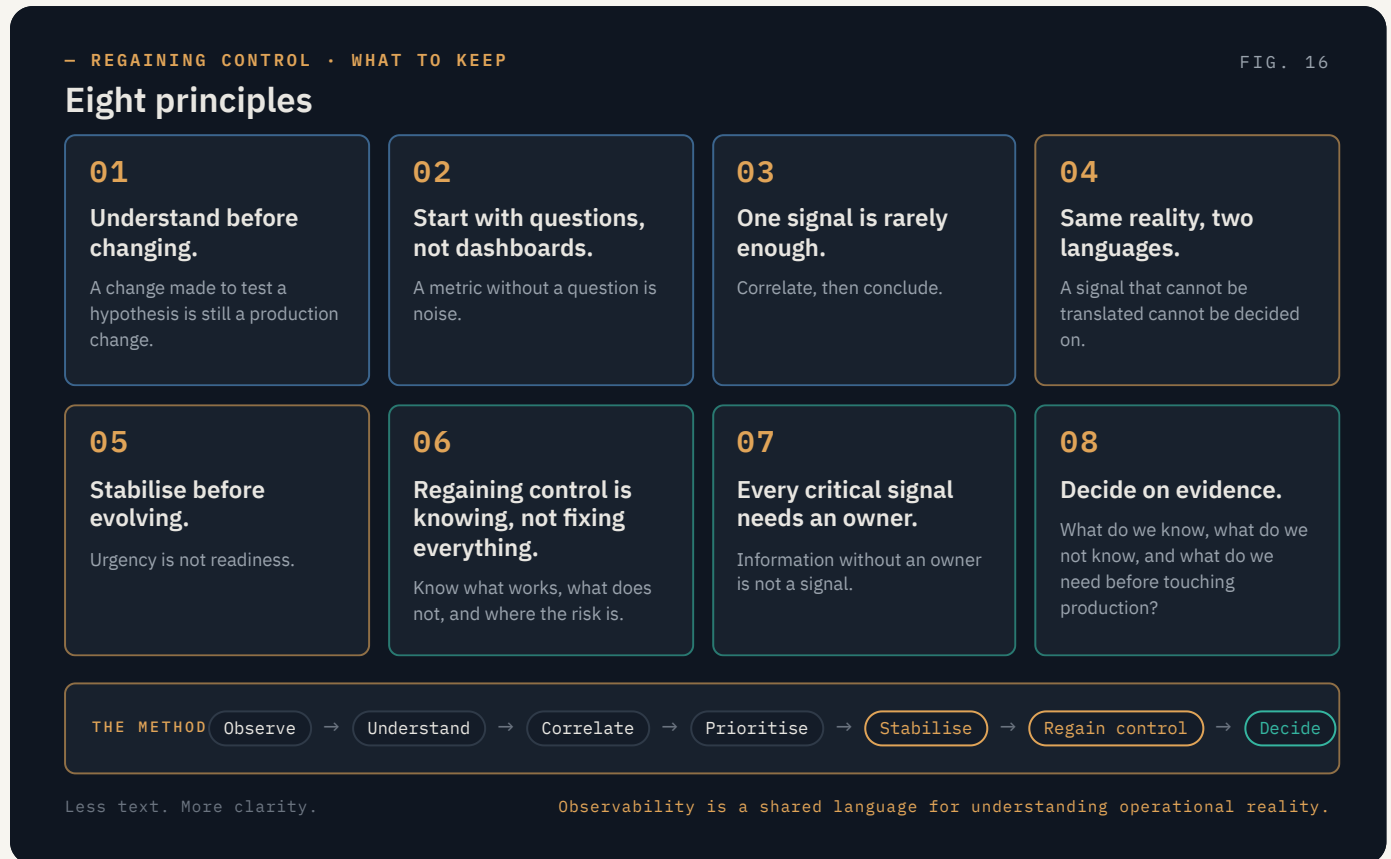
Release age, end of support, unpatched vulnerabilities and expiring certificates say how soon a change is needed. They never say the platform is ready for it. An urgent change on an unstable platform is two incidents, not one.

What "safely" means

Not "without risk", but with a risk that is known, accepted by someone who can accept it, bounded by a rollback, and observed with the same signals once the change is made.

Eight principles

Less text. More clarity.



The principles

1. **Understand before changing.** A change made to test a hypothesis is still a production change.
2. **Start with questions, not dashboards.** A metric without a question is noise.
3. **One signal is rarely enough.** Correlate, then conclude.
4. **Same reality, two languages.** A signal that cannot be translated cannot be decided on.
5. **Stabilise before evolving.** Urgency is not readiness.
6. **Regaining control is knowing, not fixing everything.** Know what works, what does not, and where the risk is.
7. **Every critical signal needs an owner.** Information without an owner is not a signal.
8. **Decide on evidence.** What do we know, what do we not know, and what do we need before touching production?

Let's talk

- 01 Is your OpenStack platform becoming difficult to understand, operate or evolve?
- 02 Do you know which signals your team is reading, and which it is not?
- 03 Do you know what you can safely change?

Before changing the platform, understand it.

If you want to read your own platform through these ten signals, I am glad to talk.

THE REPOSITORY

The source of every signal, the selection criteria, the signals left out, and how this guide is built.

github.com/soufian-zaouam/openstack-regaining-control

THE BOOK

OpenStack, the Day After Tomorrow — why control over a platform erodes, and how to keep it.

github.com/soufian-zaouam/openstack-the-day-after-tomorrow

THE PRODUCTION GUIDE

The checks, commands and anonymised incident cases behind these signals.

github.com/soufian-zaouam/openstack-production-guide

LINKEDIN

For the conversation.

www.linkedin.com/in/soufian-zaouam

OBSERVABILITY IS A SHARED
LANGUAGE FOR UNDERSTANDING
OPERATIONAL REALITY.

Observe

→

Understand

→

Correlate

→

Prioritise

→

Stabilise

→

Regain control

→

Decide

Soufian Zaouam

Platform engineer, Day-2 operations of mission-critical OpenStack platforms; author of "OpenStack, the Day After Tomorrow".

www.linkedin.com/in/soufian-zaouam · github.com/soufian-zaouam

Version 1.0 · 2026-09-29. Licensed under Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0).

An independent work. Not affiliated with, sponsored by or endorsed by the OpenInfra Foundation or any OpenStack project team. OpenStack is a trademark of the OpenStack Foundation d/b/a Open Infrastructure Foundation.